atec

Python

Ficha trabalho 0

Dict

Registo de Códigos Postais

Pretende-se desenvolver um programa em Python que permita ao utilizador registar três conjuntos de dados relacionados com códigos postais.

Cada registo deve conter as seguintes informações:

- Localidade
- Morada
- CP4 (os primeiros 4 dígitos do código postal)
- CP3 (os últimos 3 dígitos do código postal)

Requisitos do programa:

1) Criar um ***dicionário*** para armazenar temporariamente os dados de cada registo.
2) Utilizar um ***ciclo*** for para pedir os dados ao utilizador três vezes.
3) Guardar cada conjunto de dados numa ***lista de dicionários***.
4) Após o preenchimento, ***imprimir a lista completa***.
5) Em seguida, percorrer a lista e ***imprimir cada registo de forma formatada*** com f-strings.

Média de um aluno

Crie uma programa que leia o ***nome*** e a ***média*** de um aluno, guardando tambem a ***situação*** num dicionario.

No final mostre o conteudo da estrutura no ecran

Jogo de dados

Criar uma aplicação onde 4 jogadores laçem um dado e tenham resultados aleatorios.

Guarde esses resultados num dicionario.

No final colocar esse dicionario em ordem, sabendo que o vencedor tirou o maior numero no dado.

Registo e Atualização de Utentes

Crie um programa em Python que permita ao utilizador registar um número variável de utentes e atualizar a informação de um deles. O programa deverá cumprir os seguintes requisitos:

Requisitos:

1) Perguntar ao utilizador quantos utentes pretende registar.

2) Para cada utente, solicitar:

    - O nome
    - O ano de nascimento (para cálculo automático da idade)
    - Uma informação adicional

3) Calcular a idade com base no ano atual (utilize datetime.now().year).

4) Guardar os dados de cada utente num dicionário e guardar todos os registos numa lista de dicionários.

5) Após o registo, apresentar todos os utentes registados, com os seus dados devidamente formatados.

6) Solicitar a atualização da informação adicional do segundo utente registado.

7) Apresentar novamente a lista completa com os dados atualizados.

Exemplo de execução

```
Quantos utentes deseja registar? 2
Digite o nome do 1º utente: Ana
Digite o ano de nascimento: 1990
Digite alguma informação adicional: Diabética
Digite o nome do 2º utente: João
Digite o ano de nascimento: 1985
Digite alguma informação adicional: Alergias a pó

Lista de utentes:
Nome: Ana, Idade: 35, Informação: Diabética
Nome: João, Idade: 40, Informação: Alergias a pó

Atualize informação do utente 2: Sem alergias

Nome: Ana, Idade: 35, Informação: Diabética
Nome: João, Idade: 40, Informação: Sem alergias
```

Estatísticas de Golos de um Jogador

Desenvolva um programa em Python que registe o número de golos marcados por um jogador ao longo de vários jogos.
Requisitos:

1) Criar um dicionário para armazenar as informações do jogador:
- Nome
- Lista com o número de golos por jogo
- Total de golos

2) Solicitar ao utilizador:
- O nome do jogador
- O número de jogos em que ele participou
- O número de golos marcados em cada jogo

3) Armazenar a lista de golos por jogo numa lista e associá-la ao dicionário com a chave "golos".

4) Calcular o total de golos marcados usando a função sum() e guardar no dicionário com a chave "total_de_golos".

5) Imprimir:
- O conteúdo completo do dicionário (apenas para verificação de estrutura).
- A lista de golos por jogo.
- Um resumo com o nome do jogador, número de jogos e total de golos.
- O número de golos feitos em cada jogo, usando um for com enumerate().

Estatísticas de Golos de um Jogador

Exemplo de execução

```
Nome do jogador: Rafael
Quantas jogos o Rafael fez? 3
Quantos golos no 1º jogo? 2
Quantos golos no 2º jogo? 0
Quantos golos no 3º jogo? 1
```

Saida esperada:

```
 =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =
{'nome': 'Rafael', 'golos': [2, 0, 1], 'total_de_golos': 3}
 =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =
[2, 0, 1]
 =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =
O jogador Rafael fez 3 jogos e marcou 3 golos.
 =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =  =
No 1º jogo fez 2 golos.
No 2º jogo fez 0 golos.
No 3º jogo fez 1 golos.
```

Estatísticas de Golos de um Jogador

Desafio extra:

Altere o codigo para inclua vários jogadores;

Inclua um sistema para visualizar a performance de cada jogador

Registo de Pessoas com Validação de Dados

Crie um programa em Python que permita ao utilizador registar informações de várias pessoas, incluindo o nome, o sexo e a idade.

O programa deverá funcionar de forma interativa, permitindo o registo de múltiplas pessoas até que o utilizador opte por parar.

Registo de Pessoas com Validação de Dados

Requisitos do programa:

1) Criar uma lista para guardar os dados de várias pessoas.

2) Criar um dicionário para guardar temporariamente os dados de cada pessoa.

3) Num ciclo while:

- Solicitar ao utilizador:
  - Nome da pessoa
  - Sexo, validando que seja apenas 'M' ou 'F' (maiúsculo ou minúsculo, mas considerar só a primeira letra).
  - Idade (número inteiro)
- Adicionar uma cópia do dicionário da pessoa à lista principal.
- Perguntar ao utilizador se deseja continuar (S ou N), com validação da resposta.

4) O ciclo deve terminar quando o utilizador responder 'N'.

5) No final, apresentar a lista completa de pessoas registadas.

Exemplo de execução:

```
Nome: Ana
Sexo: f
Idade: 25
Quer continuar? s

Nome: Miguel
Sexo: M
Idade: 32
Quer continuar? n
```

Saida

```
[{'nome': 'Ana', 'sexo': 'F', 'idade': 25}, {'nome': 'Miguel', 'sexo': 'M', 'idade': 32}]
```

Registo de Pessoas com Validação de Dados

Desafio extra:

- Contar quantas pessoas foram registadas.
- Calcular a média de idade.
- Listar apenas os nomes das mulheres registadas.

Exercicio 1: Notas de alunos

Crie um programa em Python que utilize um dicionário (dict) para guardar nomes de alunos e as suas respetivas notas finais.

O programa deve seguir os seguintes passos:

1) Solicitar ao utilizador quantos alunos deseja registar.

2) Para cada aluno, pedir o nome e a nota (valor numérico entre 0 e 20).

3) Armazenar os dados num dicionário, onde a chave é o nome do aluno e o valor é a sua nota.

4) No final, o programa deve apresentar:

- A lista completa de alunos com as respetivas notas.
- A média das notas.
- O nome(s) do(s) aluno(s) com a nota mais alta.

Exercicio 2: Automovel

Crie uma aplicação para gerir uma coleção de automóveis utilizando um dicionário, onde cada automóvel é identificado por uma matrícula única (ex: "AB-12-XY").

Cada entrada no dicionário deve armazenar os seguintes dados do automóvel:

- Marca (ex: "Toyota")
- Modelo (ex: "Corolla")
- Ano (ex: 2020)
- Preço (ex: 18000.00)

Requisitos Técnicos:Utilizar um dicionário principal, onde a chave é a matrícula e o valor é outro dicionário com os dados do automóvel.Utilizar funções separadas para cada funcionalidade.Incluir validações básicas (ex: matrícula repetida, matrícula inexistente, tipo de dados).

Exercicio 2: Automovel

Implemente um menu interativo com as seguintes funcionalidades:

1) Adicionar um novo automóvel
   - O utilizador insere a matrícula e os restantes dados do automóvel.
   - Não permitir a duplicação de matrículas.
2) Pesquisar automóvel por matrícula
   - O utilizador insere a matrícula e o programa mostra os dados correspondentes (se existir).
3) Apagar automóvel por matrícula
   - O utilizador insere a matrícula e o programa remove o automóvel (se existir).
4) Alterar os dados de um automóvel
   - O utilizador insere a matrícula e pode modificar os dados (marca, modelo, ano e preço).
5) Listar todos os automóveis registados
   - Mostrar no ecrã todas as entradas no dicionário, com formatação legível.
6) Sair da aplicação